"House Price Prediction using Data Analysis"

WEEKLY REPORT

Submitted by ***team 3****:*

HARIHARAN VS – 24BAD030

JEEVAPRAKASH S – 24BAD048

JESSICA SAM – 24BAD049

KAVIN T – 24BAD058

24ADI204 - DATA SCIENCE AND VISUALISATION

Faculty Name: Shriaarthy E

APRIL 2026

WEEK 1 – TEAM FORMATION & DATA HUNTING

Interactive Python GitHub/Kaggle Link:
[**Github Repo Link**]

SDG 11 – Sustainable Cities and Communities

Problem Statement:

The Housing Market Bubble under Finance, Banking and Economics category.

Problem Abstract:

This project analyzes a housing price dataset to identify the factors that influence house prices. Missing values are handled, and multiple area-related columns are combined into a single Total Square Feet feature using feature engineering.

Dataset Chosen:

https://www.kaggle.com/datasets/juhibhojani/house-price/data

Team Contributions:

Name	Roll No.	Contribution
Hariharan V S	24BAD030	Created the Colab File
Jessica Sam B	24BAD049	Created the Report (word document)
Jeevaprakash S	24BAD048	Created the Git repository
Kavin T	24BAD058	Analyzed the Dataset

Learnings / Work Done:

- Explored and selected a suitable dataset based on project requirements and data quality from Kaggle.
- Created and set up a GitHub repository to manage project files and code.
- NumPy & Pandas: Learned the basics of NumPy for numerical operations using arrays, indexing, and mathematical functions. Explored Pandas for data manipulation using Series and DataFrames. Key functions used: array(), mean(), shape(), read_csv(), head(), info(), describe(), isnull().

WEEK 2 – Know Your Data

Dataset Overview

- Data Type: Pandas DataFrame
- Total Rows: 187,531
- Total Columns: 21

This is a real-estate housing dataset collected from a major Indian property portal. It has 187,531 records with 21 features — a mix of numerical and categorical data like property price, carpet area, furnishing status, ownership type, floor info, and amenities. Since the data comes from actual listings, it naturally has some inconsistencies we had to work around.

Library Setup

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style='whitegrid', palette='muted')
plt.rcParams['figure.figsize'] = (12, 6)
plt.rcParams['font.size'] = 12

print('Libraries loaded successfully.')
```

Output:

Libraries loaded sucessfully

Loading the Dataset

When we first tried loading the CSV with the default pandas settings, it threw an error. The dataset had unclosed quotation marks and multiline text inside description fields, which the default C engine couldn't handle.

Code:

```
# Added engine="python" and on_bad_lines="skip" to handle messy CSV
df = pd.read_csv('/content/house_prices.csv', engine='python', on_bad_lines='skip')
print(f'Dataset Shape: {df.shape[0]} rows x {df.shape[1]} columns')
print(f'Memory Usage: {df.memory_usage(deep=True).sum() / 1e6:.2f} MB')
df.head()
```

Output:

	Index	Title	Description	Amount(in rupees)	Price (in rupees)	location	Carpet Area	Status	Floor	Transaction	...	facing	overlooking	Society	Bathroom	Balcony	Car Parking	Ownership	Super Area	Dimensions	Plot Area
0	0	1 BHK Ready to Occupy Flat for sale in Srushti...	Bhiwandi, Thane has an attractive 1 BHK Flat f...	42 Lac	6000.0	thane	500 sqft	Ready to Move	10 out of 11	Resale	...	NaN	NaN	Srushti Siddhi Mangal Murti Complex	1	2	NaN	NaN	NaN	NaN	NaN
1	1	2 BHK Ready to Occupy Flat for sale in Dosti V...	One can find this stunning 2 BHK flat for sale...	98 Lac	13799.0	thane	473 sqft	Ready to Move	3 out of 22	Resale	...	East	Garden/Park	Dosti Vihar	2	NaN	1 Open	Freehold	NaN	NaN	NaN
2	2	2 BHK Ready to Occupy Flat for sale in Sunrise...	Up for immediate sale is a 2 BHK apartment in ...	1.40 Cr	17500.0	thane	779 sqft	Ready to Move	10 out of 29	Resale	...	East	Garden/Park	Sunrise by Kalpataru	2	NaN	1 Covered	Freehold	NaN	NaN	NaN
3	3	1 BHK Ready to Occupy Flat for sale Kasheli	This beautiful 1 BHK Flat is available for sal...	25 Lac	NaN	thane	530 sqft	Ready to Move	1 out of 3	Resale	...	NaN	NaN	NaN	1	1	NaN	NaN	NaN	NaN	NaN
4	4	2 BHK Ready to Occupy Flat for sale in TenX Ha...	This lovely 2 BHK Flat in Pokhran Road, Thane ...	1.60 Cr	18824.0	thane	635 sqft	Ready to Move	20 out of 42	Resale	...	West	Garden/Park, Main Road	TenX Habitat Raymond Realty	2	NaN	1 Covered	Co-operative Society	NaN	NaN	NaN

Pandas has two CSV parsers — the C engine (default, fast but strict) and the Python engine (slower but more flexible). Our dataset had unclosed quotes and irregular rows, so we had to switch to the Python engine. on_bad_lines="skip" skips rows that can't be parsed, so we lose a few records but the rest loads cleanly.

Feature Summary

Code:

```
feature_info = pd.DataFrame({
    'Data Type': df.dtypes,
    'Non-Null Count': df.notnull().sum(),
    'Null Count': df.isnull().sum(),
    'Null %': (df.isnull().sum() / len(df) * 100).round(2),
    'Unique Values': df.nunique(),
    'Sample Value': df.iloc[0]
})
print(feature_info.to_string())
```

Output:

	Data Type	Non-Null Count	Null Count	Null %	Unique Values	Sample Value
Index	int64	23558	0	0.00	23558	0
Title	object	23558	0	0.00	5535	1 BHK Ready to Occupy Flat for sale in Srushti...
Description	object	23027	531	2.25	8255	Bhiwandi, Thane has an attractive 1 BHK Flat f...
Amount(in rupees)	object	23558	0	0.00	866	42 Lac
Price (in rupees)	float64	22393	1165	4.95	4469	6000.0
location	object	23558	0	0.00	6	thane
Carpet Area	object	11123	12435	52.78	1272	500 sqft
Status	object	23517	41	0.17	1	Ready to Move
Floor	object	22672	886	3.76	716	10 out of 11
Transaction	object	23546	12	0.05	4	Resale
Furnishing	object	23098	460	1.95	3	Unfurnished
facing	object	12360	11198	47.53	8	NaN
overlooking	object	11552	12006	50.96	14	NaN
Society	object	8901	14657	62.22	1977	Srushti Siddhi Mangal Murti Complex
Bathroom	object	23511	47	0.20	11	1
Balcony	object	14799	8759	37.18	9	2
Car Parking	object	9030	14528	61.67	62	NaN
Ownership	object	13312	10246	43.49	4	NaN
Super Area	object	12253	11305	47.99	1241	NaN
Dimensions	float64	0	23558	100.00	0	NaN
Plot Area	float64	0	23558	100.00	0	NaN

Key things we noticed from this:

- Amount(in rupees) has values like "42 Lac", "1.40 Cr" — stored as text, needs parsing
- Carpet Area, Super Area, Plot Area contain "500 sqft" format — also needs cleaning
- Dimensions and Plot Area are almost entirely null (>80%) — probably need to drop them
- Price (in rupees) is already numeric — this is price per sqft

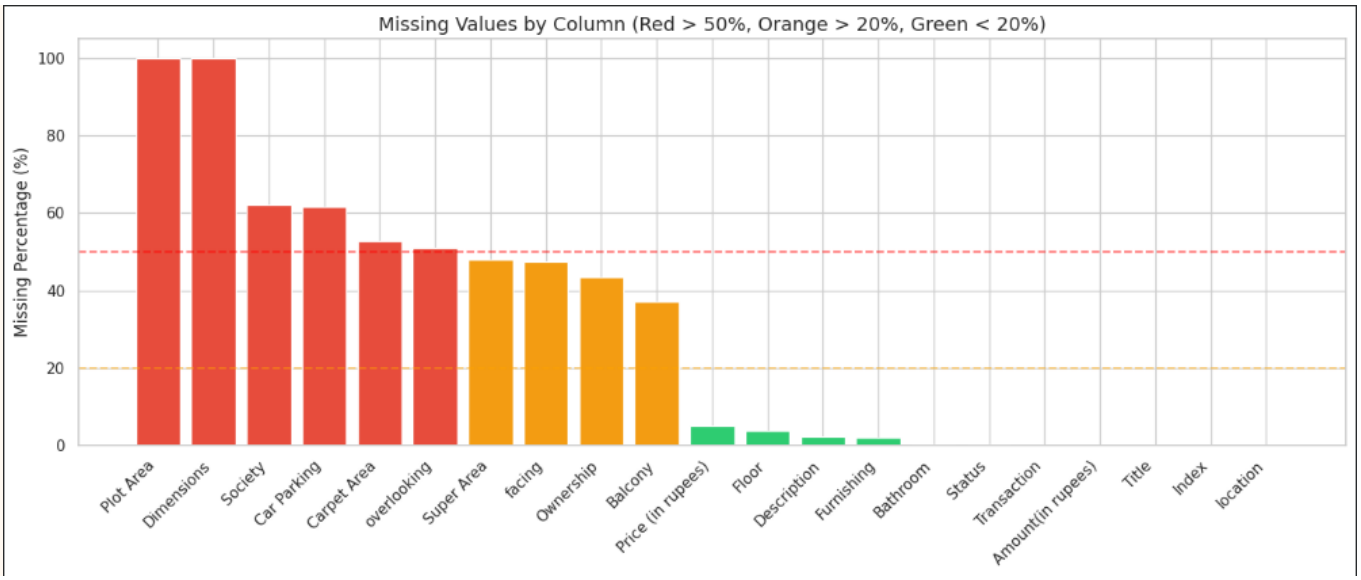
Missing Values Check

Code:

```
# Duplicate check
print(f'Duplicate rows: {df.duplicated().sum()}')

# Missing values bar chart
fig, ax = plt.subplots(figsize=(14, 6))
missing = df.isnull().sum().sort_values(ascending=False)
missing_pct = (missing / len(df) * 100).round(2)
colors = ['#e74c3c' if x > 50 else '#f39c12' if x > 20 else '#2ecc71' for x in missing_pct]
ax.bar(range(len(missing)), missing_pct, color=colors)
ax.set_xticks(range(len(missing)))
ax.set_xticklabels(missing.index, rotation=45, ha='right')
ax.set_ylabel('Missing Percentage (%)')
ax.set_title('Missing Values by Column')
ax.axhline(y=50, color='red', linestyle='--', alpha=0.5)
ax.axhline(y=20, color='orange', linestyle='--', alpha=0.5)
plt.tight_layout()
```

```
plt.show()
```

Output:



From the chart: Dimensions and Plot Area are >80% missing (red bars). Facing, Overlooking, Society, Car Parking are moderate (orange). Price, Carpet Area, Floor are mostly available (green). This told us which columns we could keep vs drop in the next week.

Team Contribution:

Name	Roll No.	Contribution
Hariharan V S	24BAD030	Report Making
Jessica Sam B	24BAD049	Code Preparing
Jeevaprakash S	24BAD048	Report Making
Kavin T	24BAD058	Code Preparing

Reference:

https://stackoverflow.com/questions/52105659/pandas-read-csv-unexpected-end-of-data-error

WEEK 3 – The Cleaning Sprint

This week was mainly about getting the data into a usable shape. We handled missing values, parsed all the weird text-based columns into proper numbers, and then did outlier detection. There was a lot of trial and error — particularly with the floor and amount columns.

Step 1 – Dropping Useless Columns

Code:

```
cols_to_drop = ['Index', 'Title', 'Description', 'Dimensions', 'Plot Area']
df_clean = df.drop(columns=cols_to_drop)
print(f'Dropped: {cols_to_drop}')
print(f'Remaining columns: {df_clean.shape[1]}')
```

Output:

```
••• Dropped columns: ['Index', 'Title', 'Description', 'Dimensions', 'Plot Area']
    Remaining columns: 16
    Remaining shape: (23558, 16)
```

Dimensions and Plot Area were almost entirely null so there's no point imputing them. Index is just a row number, and Title/Description are free text that we can't use for numeric analysis.

Step 2 – Parsing Mixed-Format Columns

Several columns had numbers stored as text (e.g., "42 Lac", "500 sqft", "10 out of 11"). We wrote individual parser functions for each one.

Code:

```
# Convert Amount(in rupees) — handles "Lac" and "Cr" formats
def parse_amount(val):
    if pd.isna(val) or val == 'Call for Price':
        return np.nan
    val = str(val).strip()
    try:
        if 'Cr' in val:
            return float(val.replace('Cr', '').strip()) * 100  # convert to Lac
        elif 'Lac' in val:
            return float(val.replace('Lac', '').strip())
        else:
            return float(val)
    except:
        return np.nan

df_clean['Amount_Lac'] = df_clean['Amount(in rupees)'].apply(parse_amount)
df_clean.drop(columns=['Amount(in rupees)'], inplace=True)

# Convert area columns — remove "sqft" text
def parse_area(val):
    if pd.isna(val):
        return np.nan
    val = str(val).replace('sqft', '').replace(',', '').strip()
    try:
        return float(val)
    except:
        return np.nan
```

```
df_clean['Carpet_Area_sqft'] = df_clean['Carpet Area'].apply(parse_area)
df_clean['Super_Area_sqft']  = df_clean['Super Area'].apply(parse_area)
df_clean.drop(columns=['Carpet Area', 'Super Area'], inplace=True)

# Floor: split "10 out of 11" into two columns
def parse_floor_current(val):
    if pd.isna(val): return np.nan
    val = str(val).strip()
    floor_str = val.split(' out of ')[0].strip()
    if floor_str.lower() in ['ground', 'lower basement', 'upper basement']:
        return 0
    try: return int(floor_str)
    except: return np.nan

def parse_floor_total(val):
    if pd.isna(val): return np.nan
    parts = str(val).strip().split(' out of ')
    if len(parts) == 2:
        try: return int(parts[1].strip())
        except: return np.nan
    return np.nan

df_clean['Current_Floor'] = df_clean['Floor'].apply(parse_floor_current)
df_clean['Total_Floors']  = df_clean['Floor'].apply(parse_floor_total)
df_clean.drop(columns=['Floor'], inplace=True)

# Car Parking — extract the number
def parse_parking(val):
    if pd.isna(val): return 0
    try: return int(str(val).strip().split()[0])
    except: return 0

df_clean['Car_Parking_Count'] = df_clean['Car Parking'].apply(parse_parking)
df_clean.drop(columns=['Car Parking'], inplace=True)

df_clean['Bathroom'] = pd.to_numeric(df_clean['Bathroom'], errors='coerce')
df_clean['Balcony']  = pd.to_numeric(df_clean['Balcony'],  errors='coerce')
print('All conversions done.')
df_clean.dtypes
```

Output:

```
...  Conversions complete.
                                0
      Price (in rupees)   float64
               location    object
                 Status    object
            Transaction    object
             Furnishing    object
                 facing    object
            overlooking    object
                Society    object
               Bathroom   float64
                Balcony   float64
              Ownership    object
             Amount_Lac   float64
        Carpet_Area_sqft  float64
         Super_Area_sqft  float64
           Current_Floor  float64
            Total_Floors  float64
       Car_Parking_Count    int64

      dtype: object
```

Step 3 – Filling Missing Values

Code:

```
# Numeric → fill with median (more robust than mean for skewed data)
for col in df_clean.select_dtypes(include=[np.number]).columns:
    if df_clean[col].isnull().sum() > 0:
        df_clean[col].fillna(df_clean[col].median(), inplace=True)
        print(f'  {col} filled with median: {df_clean[col].median():.2f}')

# Categorical → fill with mode
for col in df_clean.select_dtypes(include=['object']).columns:
    if df_clean[col].isnull().sum() > 0:
        df_clean[col].fillna(df_clean[col].mode()[0], inplace=True)
        print(f'  {col} filled with mode: {df_clean[col].mode()[0]}')

print(f'Remaining nulls: {df_clean.isnull().sum().sum()}')
```

Output:

```
...                        Count  Percentage
     Carpet_Area_sqft      14977       63.58
     Society               14657       62.22
     Super_Area_sqft       12965       55.03
     overlooking           12006       50.96
     facing                11198       47.53
     Ownership             10246       43.49
     Balcony                8764       37.20
     Price (in rupees)      1165        4.95
     Total_Floors            890        3.78
     Current_Floor           886        3.76
     Amount_Lac              489        2.08
     Furnishing              460        1.95
     Bathroom                 55        0.23
     Status                   41        0.17
     Transaction              12        0.05
     Filled Price (in rupees) with median: 5576.0
     Filled Bathroom with median: 2.0
     Filled Balcony with median: 2.0
     Filled Amount_Lac with median: 78.0
     Filled Carpet_Area_sqft with median: 900.0
     Filled Super_Area_sqft with median: 1332.0
     Filled Current_Floor with median: 3.0
     Filled Total_Floors with median: 7.0
     Filled Status with mode: Ready to Move
     Filled Transaction with mode: Resale
     Filled Furnishing with mode: Unfurnished
     Filled facing with mode: East
     Filled overlooking with mode: Main Road
     Filled Society with mode: Nebula Tower
     Filled Ownership with mode: Freehold

     Remaining nulls: 0
```

We used median for numbers because price and area data is right-skewed — the mean gets pulled by expensive outliers. For categories we just used the most common value.

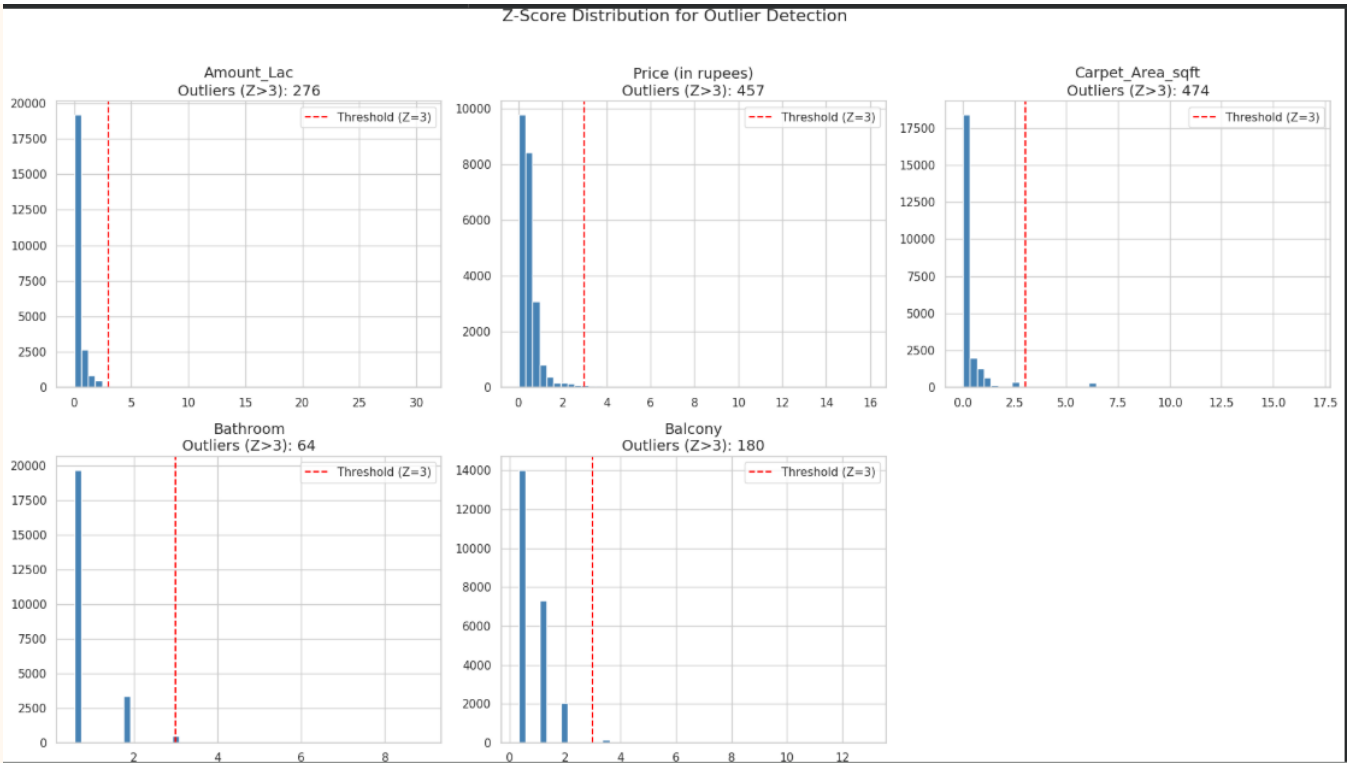
Step 4 – Outlier Detection

Code – Z-Score:

```
from scipy import stats

key_numeric = ['Amount_Lac', 'Price (in rupees)', 'Carpet_Area_sqft', 'Bathroom', 'Balcony']
key_numeric = [c for c in key_numeric if c in df_clean.columns]

fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.flatten()
for i, col in enumerate(key_numeric):
    z_scores = np.abs(stats.zscore(df_clean[col].dropna()))
    outlier_count = (z_scores > 3).sum()
    axes[i].hist(z_scores, bins=50, color='steelblue', edgecolor='white')
    axes[i].axvline(x=3, color='red', linestyle='--', label='Z=3 threshold')
    axes[i].set_title(f'{col}  |  Outliers: {outlier_count}')
    axes[i].legend()
```

```
for j in range(len(key_numeric), 6):
    axes[j].set_visible(False)
plt.suptitle('Z-Score Distribution - Outlier Detection', fontsize=15)
plt.tight_layout()
plt.show()
```

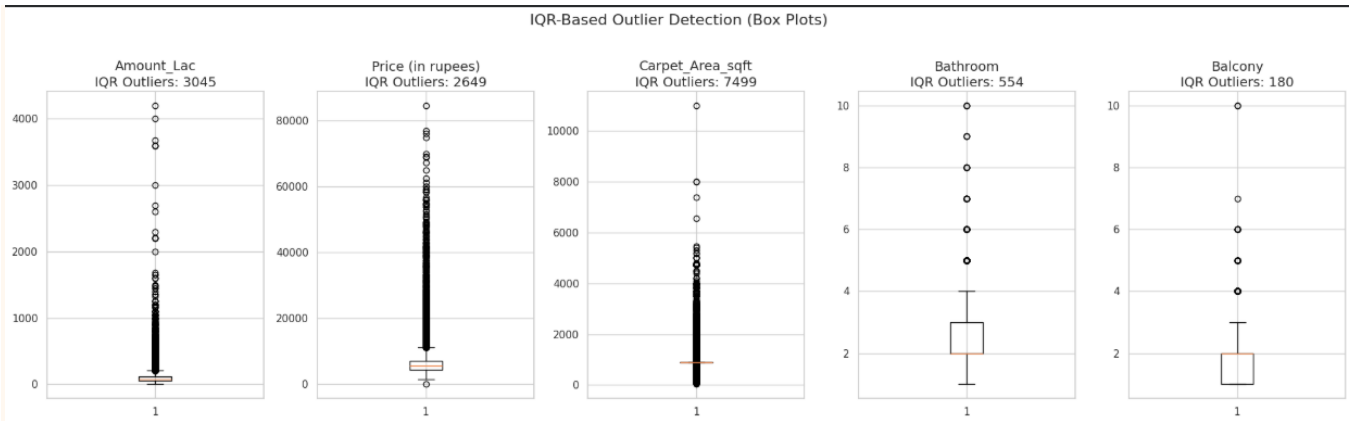
Output:



Code – IQR Boxplots:

```
fig, axes = plt.subplots(1, len(key_numeric), figsize=(4*len(key_numeric), 6))
for i, col in enumerate(key_numeric):
    Q1 = df_clean[col].quantile(0.25)
    Q3 = df_clean[col].quantile(0.75)
    IQR = Q3 - Q1
    outliers = ((df_clean[col] < Q1 - 1.5*IQR) | (df_clean[col] > Q3 + 1.5*IQR)).sum()
    axes[i].boxplot(df_clean[col].dropna(), vert=True)
    axes[i].set_title(f'{col}\nIQR outliers: {outliers}')
plt.suptitle('IQR Boxplots', fontsize=15)
plt.tight_layout()
plt.show()
```

Visualization Output:



Interpretation:

- Index – widely spread but irrelevant (just a row counter), no analytical use
- Price – median is around ₹6,000–7,000 per sqft. Lots of high-value outliers up to ~₹85,000. The distribution is very right-skewed — expected in property data.
- Amount_Lac and Carpet_Area_sqft also show similar right-skew patterns.

We decided not to remove outliers — they represent real premium listings and removing them would lose useful data.

Data Quality Report

Code:

```
print('='*60)
print('DATA QUALITY REPORT')
print('='*60)
print(f'Total Records:        {len(df_clean)}')
print(f'Total Features:       {df_clean.shape[1]}')
print(f'Numeric Features:     {len(df_clean.select_dtypes(include=[np.number]).columns)}')
print(f'Categorical Features: {len(df_clean.select_dtypes(include=["object"]).columns)}')
print(f'Missing Values Left:  {df_clean.isnull().sum().sum()}')
print(f'Duplicate Rows:       {df_clean.duplicated().sum()}')
print('\nStats:')
print(df_clean.describe().to_string())
print('='*60)
```

Output:

```
==========
DATA QUALITY REPORT SUMMARY
==========
Total Records: 23558
Total Features: 17
Numeric Features: 9
Categorical Features: 8
Missing Values Remaining: 0
Duplicate Rows: 15044

Numeric Column Statistics:
       Price (in rupees)      Bathroom       Balcony    Amount_Lac  Carpet_Area_sqft  Super_Area_sqft  Current_Floor  Total_Floors  Car_Parking_Count
count       23558.000000  23558.000000  23558.000000  23558.000000      23558.000000     23558.000000   23558.000000  23558.000000       23558.000000
mean         6873.627175      2.484124      1.795017    113.020307       1028.437091      1412.719586       4.709058      9.563248           0.938874
std          4884.513267      0.841710      0.634076    133.352520        589.396567       639.066749       5.115891      7.009770          13.020574
min            17.000000      1.000000      1.000000      2.000000         30.000000        27.000000       0.000000      1.000000           0.000000
25%          4500.000000      2.000000      1.000000     58.000000        900.000000      1332.000000       1.000000      4.000000           0.000000
50%          5576.000000      2.000000      2.000000     78.000000        900.000000      1332.000000       3.000000      7.000000           0.000000
75%          7170.000000      3.000000      2.000000    117.000000        900.000000      1332.000000       6.000000     13.000000           1.000000
max         84619.000000     10.000000     10.000000   4200.000000      11000.000000     40000.000000      75.000000     91.000000         706.000000
==========
```

Team Contribution:

Name	Roll No.	Contribution
Hariharan V S	24BAD030	Report Making
Jessica Sam B	24BAD049	Code Preparing
Jeevaprakash S	24BAD048	Report Making
Kavin T	24BAD058	Code Preparing

WEEK 4 – EDA Deep Dive

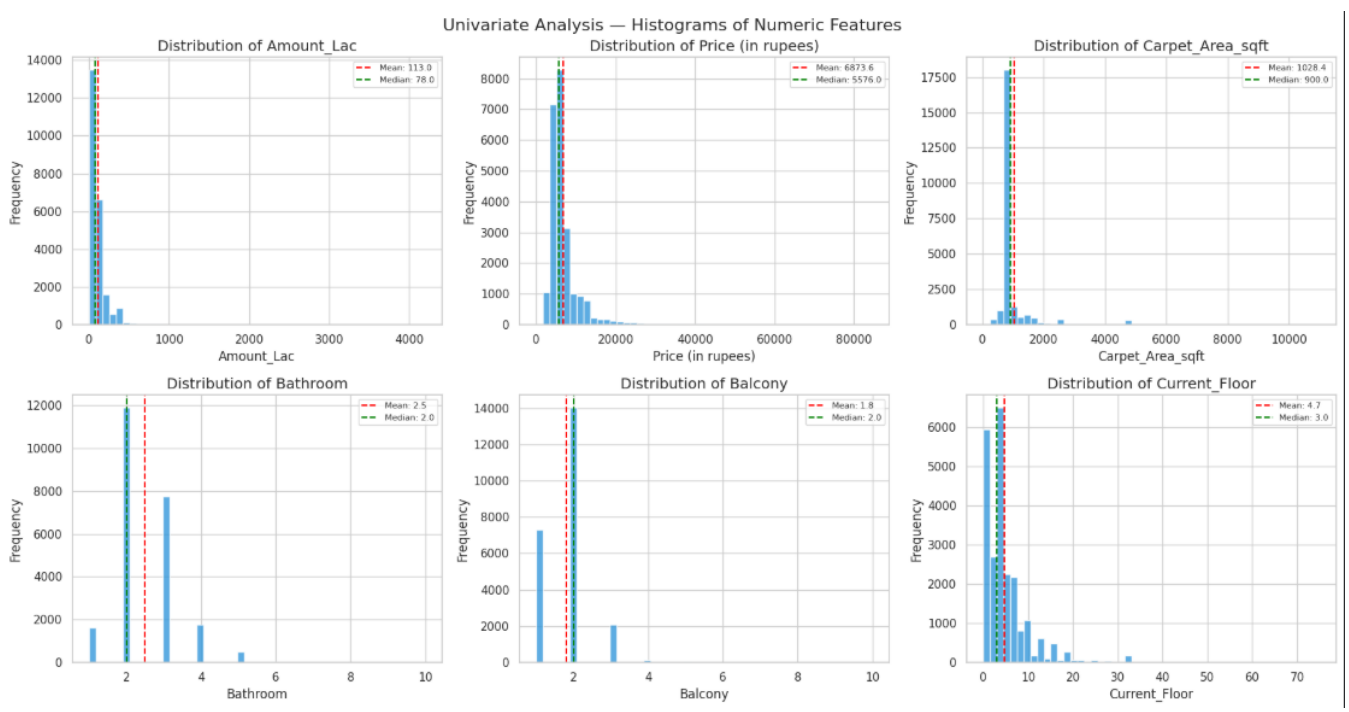
Week 4 was all about understanding the data visually. We did univariate analysis (one column at a time), bivariate (two columns), and then some multivariate stuff. This is where the interesting patterns started showing up.

4.1 Univariate Analysis – Numeric Features

Code:

```
num_features = ['Amount_Lac', 'Price (in rupees)', 'Carpet_Area_sqft',
                'Bathroom', 'Balcony', 'Current_Floor', 'Total_Floors']
num_features = [c for c in num_features if c in df_clean.columns]

fig, axes = plt.subplots(3, 3, figsize=(18, 14))
axes = axes.flatten()
for i, col in enumerate(num_features):
    axes[i].hist(df_clean[col].dropna(), bins=50, color='#3498db', edgecolor='white',
alpha=0.8)
    axes[i].axvline(df_clean[col].mean(),   color='red',   linestyle='--',
                    label=f'Mean: {df_clean[col].mean():.1f}')
    axes[i].axvline(df_clean[col].median(), color='green', linestyle='--',
                    label=f'Median: {df_clean[col].median():.1f}')
    axes[i].set_title(f'Distribution of {col}')
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('Frequency')
    axes[i].legend(fontsize=8)
for j in range(len(num_features), 9):
    axes[j].set_visible(False)
plt.suptitle('Univariate Analysis - Numeric Features', fontsize=16)
plt.tight_layout()
plt.show()
```

Visualization:



Key observations:

- Amount_Lac – heavily right-skewed, most properties under 100 Lac. A few go really high.
- Price per sqft – concentrated around 5,000–15,000 Rs/sqft with some extreme outliers.

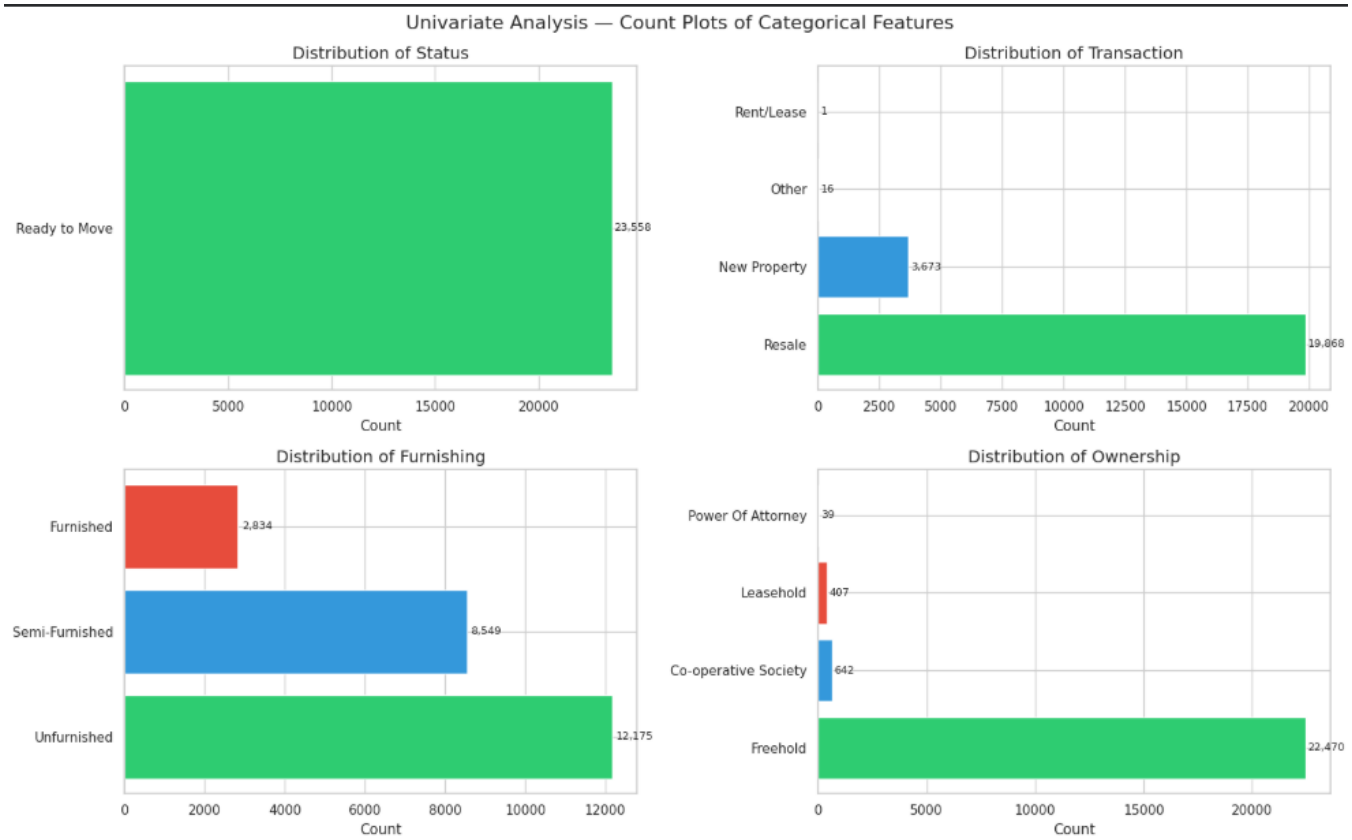
- Carpet Area – majority between 400–800 sqft, typical for 1–2 BHK flats.
- Bathroom – most properties have 1–2 bathrooms. Makes sense for the size range.
- Mean > Median across almost all columns → confirms positive skew.

4.2 Univariate Analysis – Categorical Features

Code:

```
cat_features = ['Status', 'Transaction', 'Furnishing', 'Ownership']
cat_features = [c for c in cat_features if c in df_clean.columns]

fig, axes = plt.subplots(2, 2, figsize=(16, 10))
axes = axes.flatten()
palette = ['#2ecc71', '#3498db', '#e74c3c', '#f39c12', '#9b59b6']
for i, col in enumerate(cat_features):
    counts = df_clean[col].value_counts().head(6)
    axes[i].barh(counts.index, counts.values, color=palette[:len(counts)])
    axes[i].set_title(f'Distribution of {col}')
    axes[i].set_xlabel('Count')
    for idx, val in enumerate(counts.values):
        axes[i].text(val + 100, idx, f'{val:,}', va='center', fontsize=9)
plt.suptitle('Univariate - Categorical Features', fontsize=16)
plt.tight_layout()
plt.show()
```

Visualization:



Observations:

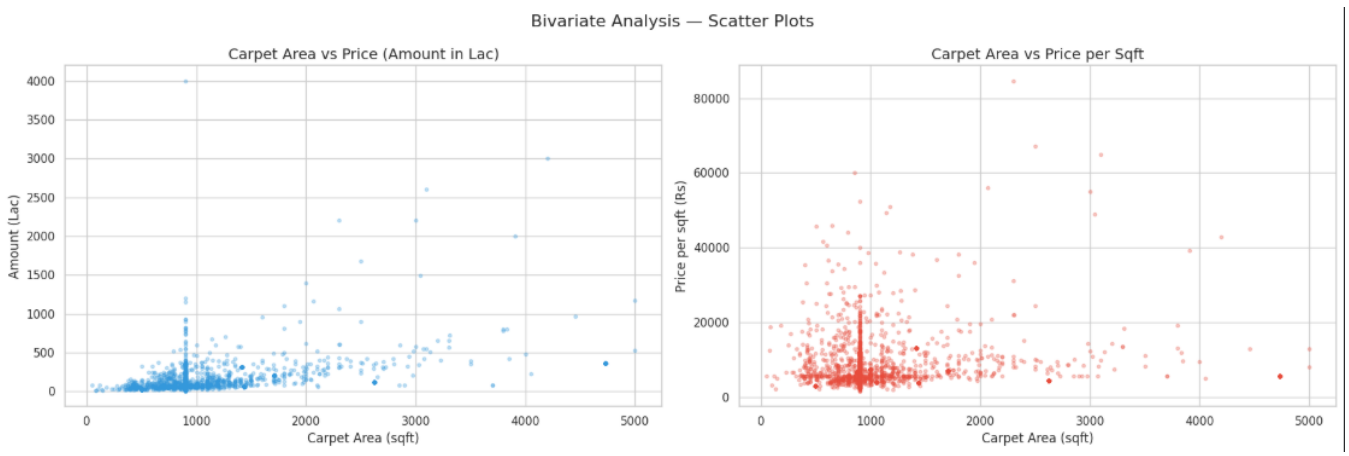
- Status – almost all "Ready to Move". Very few under construction listings.
- Transaction – mostly Resale. New property listings are a minority.
- Furnishing – Semi-Furnished is the biggest group, then Unfurnished, then Furnished.
- Ownership – Freehold is dominant. Most people prefer full ownership.

4.3 Bivariate Analysis

Code – Scatter Plots:

```
sample = df_clean.sample(min(5000, len(df_clean)), random_state=42)

fig, axes = plt.subplots(1, 2, figsize=(18, 6))
axes[0].scatter(sample['Carpet_Area_sqft'], sample['Amount_Lac'],
                alpha=0.3, s=10, c='#3498db')
axes[0].set(xlabel='Carpet Area (sqft)', ylabel='Amount (Lac)',
            title='Carpet Area vs Total Price')

axes[1].scatter(sample['Carpet_Area_sqft'], sample['Price (in rupees)'],
                alpha=0.3, s=10, c='#e74c3c')
axes[1].set(xlabel='Carpet Area (sqft)', ylabel='Price per sqft (Rs)',
            title='Carpet Area vs Price per Sqft')

plt.suptitle('Bivariate - Scatter Plots', fontsize=16)
plt.tight_layout()
plt.show()
```

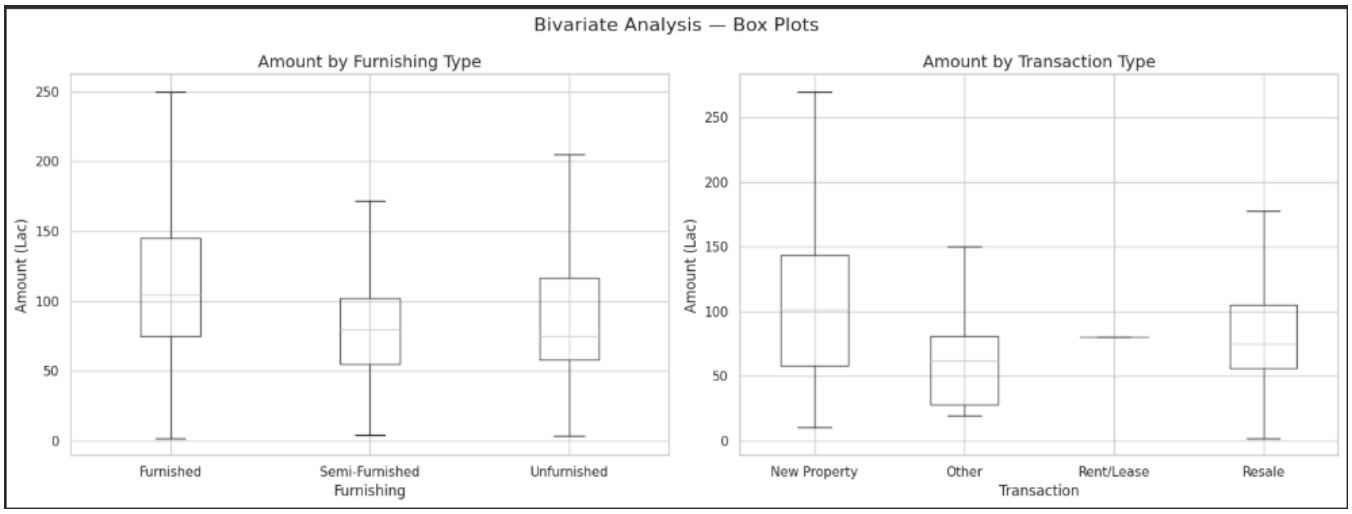
Visualization:



Code – Boxplots by Category:

```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
df_clean.boxplot(column='Amount_Lac', by='Furnishing',  ax=axes[0], showfliers=False)
df_clean.boxplot(column='Amount_Lac', by='Transaction', ax=axes[1], showfliers=False)
axes[0].set(title='Amount by Furnishing', xlabel='Furnishing', ylabel='Amount (Lac)')
axes[1].set(title='Amount by Transaction', xlabel='Transaction', ylabel='Amount (Lac)')
plt.suptitle('Bivariate - Box Plots')
plt.tight_layout()
plt.show()
```

Visualization:



What we found:

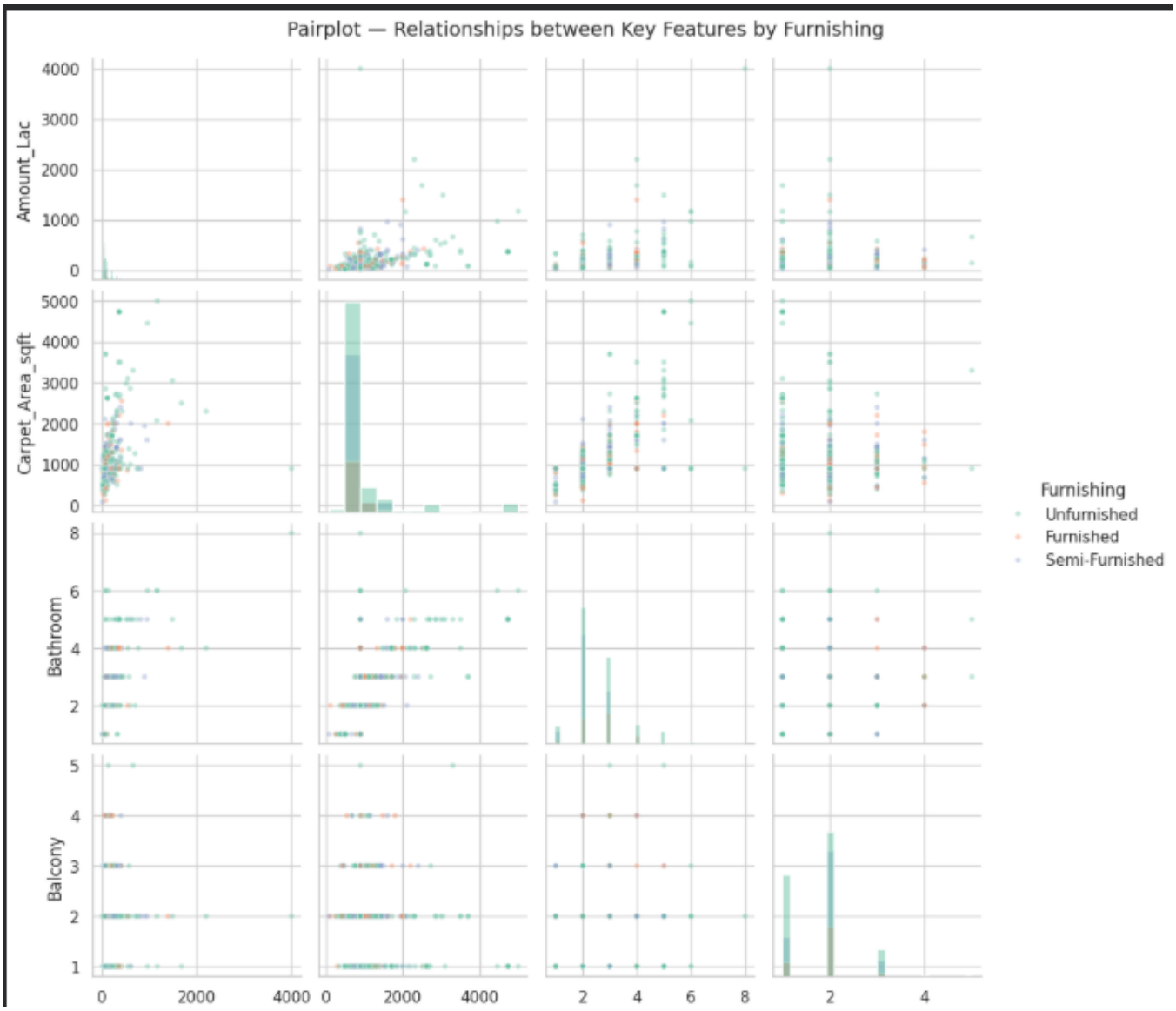
- Clear positive relationship between carpet area and total price — bigger flat = higher cost (expected).
- Price per sqft is much more scattered, showing location has a big impact beyond just size.
- Furnished properties have slightly higher median prices. New Property listings cost more than Resale.

4.4 Multivariate Analysis

Code – Pairplot:

```
pair_cols = ['Amount_Lac', 'Carpet_Area_sqft', 'Bathroom', 'Balcony', 'Furnishing']
pair_cols = [c for c in pair_cols if c in df_clean.columns]
sample_pair = df_clean[pair_cols].sample(min(2000, len(df_clean)), random_state=42).dropna()

g = sns.pairplot(sample_pair, hue='Furnishing', palette='Set2',
                 diag_kind='hist', plot_kws={'alpha': 0.4, 's': 15}, height=2.5)
g.figure.suptitle('Pairplot - Features by Furnishing Type', y=1.02, fontsize=14)
plt.show()
```

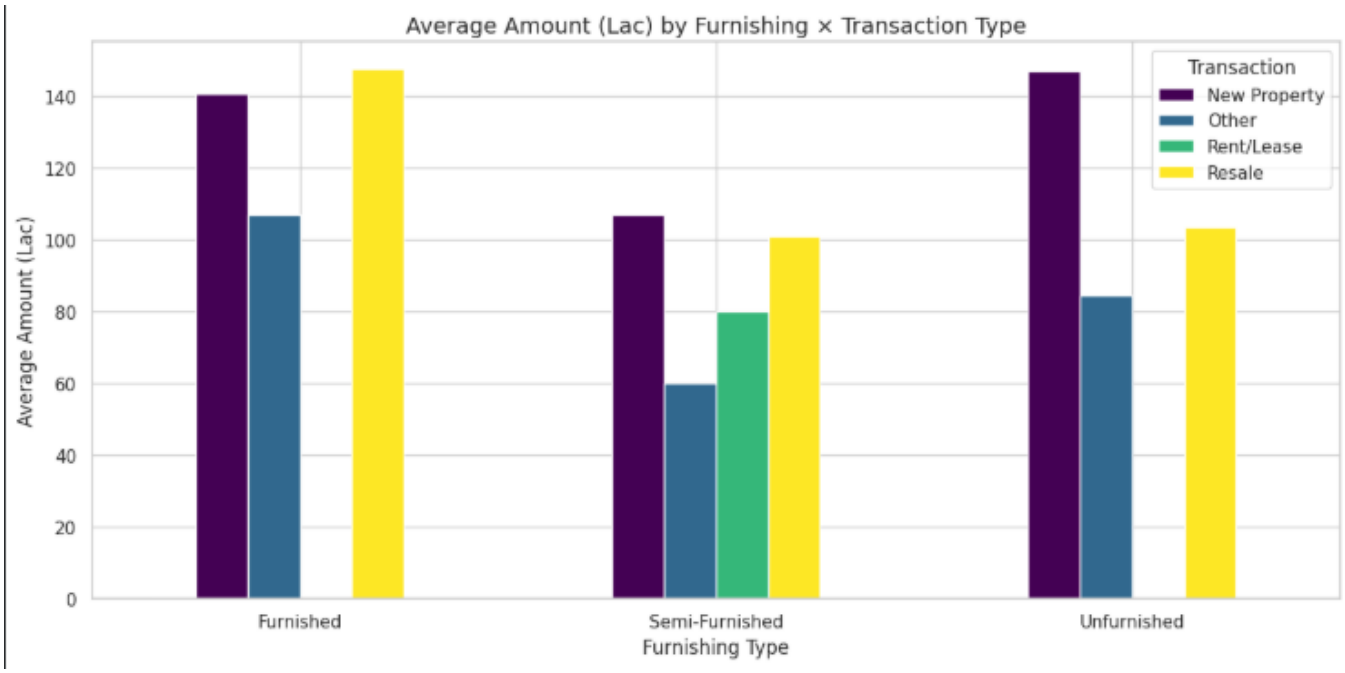
Visualization:



Code – Grouped Bar Chart:

```
fig, ax = plt.subplots(figsize=(12, 6))
grouped = df_clean.groupby(['Furnishing', 'Transaction'])['Amount_Lac'].mean().unstack()
grouped.plot(kind='bar', ax=ax, colormap='viridis', edgecolor='white')
ax.set(title='Avg Price by Furnishing × Transaction',
       ylabel='Average Amount (Lac)', xlabel='Furnishing')
plt.xticks(rotation=0)
plt.legend(title='Transaction')
plt.tight_layout()
plt.show()
```

Visualization:



From the grouped chart — Furnished + New Property is consistently the highest price segment. The gap between New and Resale is pretty clear across all furnishing types.

Team Contribution:

Name	Roll No.	Contribution
Hariharan V S	24BAD030	Code Preparing
Jessica Sam B	24BAD049	Report Making
Jeevaprakash S	24BAD048	Updating GitHub
Kavin T	24BAD058	Code Preparing

WEEK 6 – Feature Engineering

This week we focused on transforming the data so it's ready for any kind of modelling or advanced analysis. That meant encoding the text categories into numbers, scaling numeric columns, and fixing the skewness in price/area columns. We also created a few new derived features.

6.1 Encoding Categorical Variables

We used Label Encoding for columns like Status, Transaction, Furnishing, Ownership — these have a small number of categories. One-Hot Encoding was used for the "facing" column since it's purely nominal. High-cardinality columns like location and Society were dropped for now.

Code:

```
from sklearn.preprocessing import LabelEncoder

df_encoded = df_clean.copy()

# Label Encoding
le_cols = ['Status', 'Transaction', 'Furnishing', 'Ownership']
le_cols = [c for c in le_cols if c in df_encoded.columns]
for col in le_cols:
    le = LabelEncoder()
    df_encoded[col + '_LE'] = le.fit_transform(df_encoded[col].astype(str))
    print(f'{col}: {dict(zip(le.classes_, le.transform(le.classes_)))}')

# One-Hot Encoding for 'facing'
ohe_cols = ['facing']
ohe_cols = [c for c in ohe_cols if c in df_encoded.columns]
df_encoded = pd.get_dummies(df_encoded, columns=ohe_cols, drop_first=True)

# Drop high-cardinality columns we can't encode efficiently
drop_cats = ['location', 'overlooking', 'Society'] + le_cols
drop_cats = [c for c in drop_cats if c in df_encoded.columns]
df_encoded.drop(columns=drop_cats, inplace=True)

print(f'Final shape: {df_encoded.shape}')
df_encoded.head()
```

Output:

```
Status: {'Ready to Move': np.int64(0)}
Transaction: {'New Property': np.int64(0), 'Other': np.int64(1), 'Rent/Lease': np.int64(2), 'Resale': np.int64(3)}
Furnishing: {'Furnished': np.int64(0), 'Semi-Furnished': np.int64(1), 'Unfurnished': np.int64(2)}
Ownership: {'Co-operative Society': np.int64(0), 'Freehold': np.int64(1), 'Leasehold': np.int64(2), 'Power Of Attorney': np.int64(3)}

Encoded shape: (23558, 20)
```

	Price (in rupees)	Bathroom	Balcony	Amount_Lac	Carpet_Area_sqft	Super_Area_sqft	Current_Floor	Total_Floors	Car_Parking_Count	Status_LE	Transaction_LE	Furnishing_LE	Ownership_LE	facing_North	facing_North - East	facing_North - West	facing_South	facing_South - East	facing_South -West	facing_West
0	8000.0	1.0	2.0	42.0	500.0	1332.0	10.0	11.0	0	0	3	2	1	False	False	False	False	False	False	False
1	13799.0	2.0	2.0	98.0	473.0	1332.0	3.0	22.0	1	0	3	1	1	False	False	False	False	False	False	False
2	17500.0	2.0	2.0	140.0	779.0	1332.0	10.0	29.0	1	0	3	2	1	False	False	False	False	False	False	False
3	5576.0	1.0	1.0	25.0	530.0	1332.0	1.0	3.0	0	0	3	2	1	False	False	False	False	False	False	False
4	18824.0	2.0	2.0	160.0	635.0	1332.0	20.0	42.0	1	0	3	2	0	False	False	False	False	False	False	True

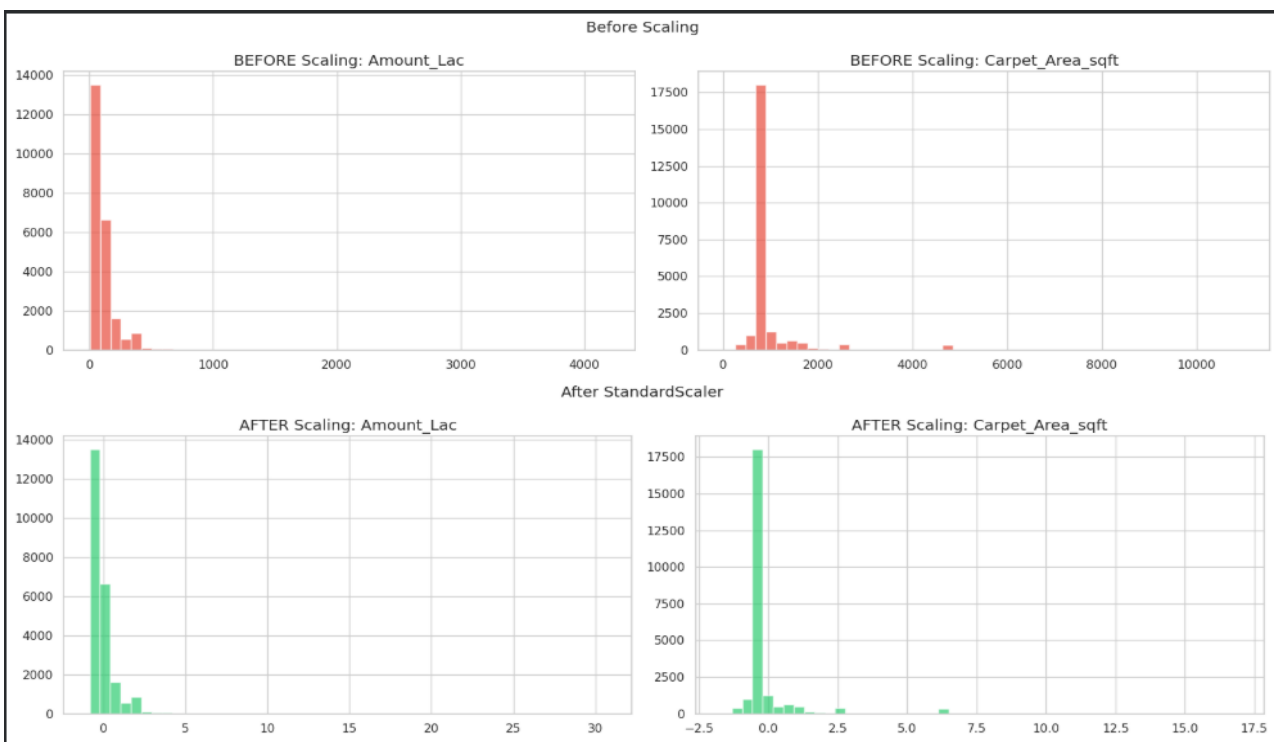
6.2 Feature Scaling

Before scaling, columns like Amount_Lac range from ~10 to 85,000+ while Bathroom ranges from 1 to 10. StandardScaler brings everything to zero mean and unit variance so no single column dominates.

Code – Before vs After:

```
from sklearn.preprocessing import StandardScaler

numeric_for_scale = df_encoded.select_dtypes(include=[np.number]).columns.tolist()
sample_cols = ['Amount_Lac', 'Carpet_Area_sqft']
sample_cols = [c for c in sample_cols if c in df_encoded.columns]

# Before
fig, axes = plt.subplots(1, 2, figsize=(16, 5))
for i, col in enumerate(sample_cols):
    axes[i].hist(df_encoded[col], bins=50, color='#e74c3c', alpha=0.7, edgecolor='white')
    axes[i].set_title(f'BEFORE Scaling: {col}')
plt.suptitle('Before Scaling')
plt.tight_layout()
plt.show()

# Apply StandardScaler
scaler = StandardScaler()
df_scaled = df_encoded.copy()
df_scaled[numeric_for_scale] = scaler.fit_transform(df_encoded[numeric_for_scale])

# After
fig, axes = plt.subplots(1, 2, figsize=(16, 5))
for i, col in enumerate(sample_cols):
    axes[i].hist(df_scaled[col], bins=50, color='#2ecc71', alpha=0.7, edgecolor='white')
    axes[i].set_title(f'AFTER Scaling: {col}')
plt.suptitle('After StandardScaler')
plt.tight_layout()
plt.show()
```

Visualization – Before & After:



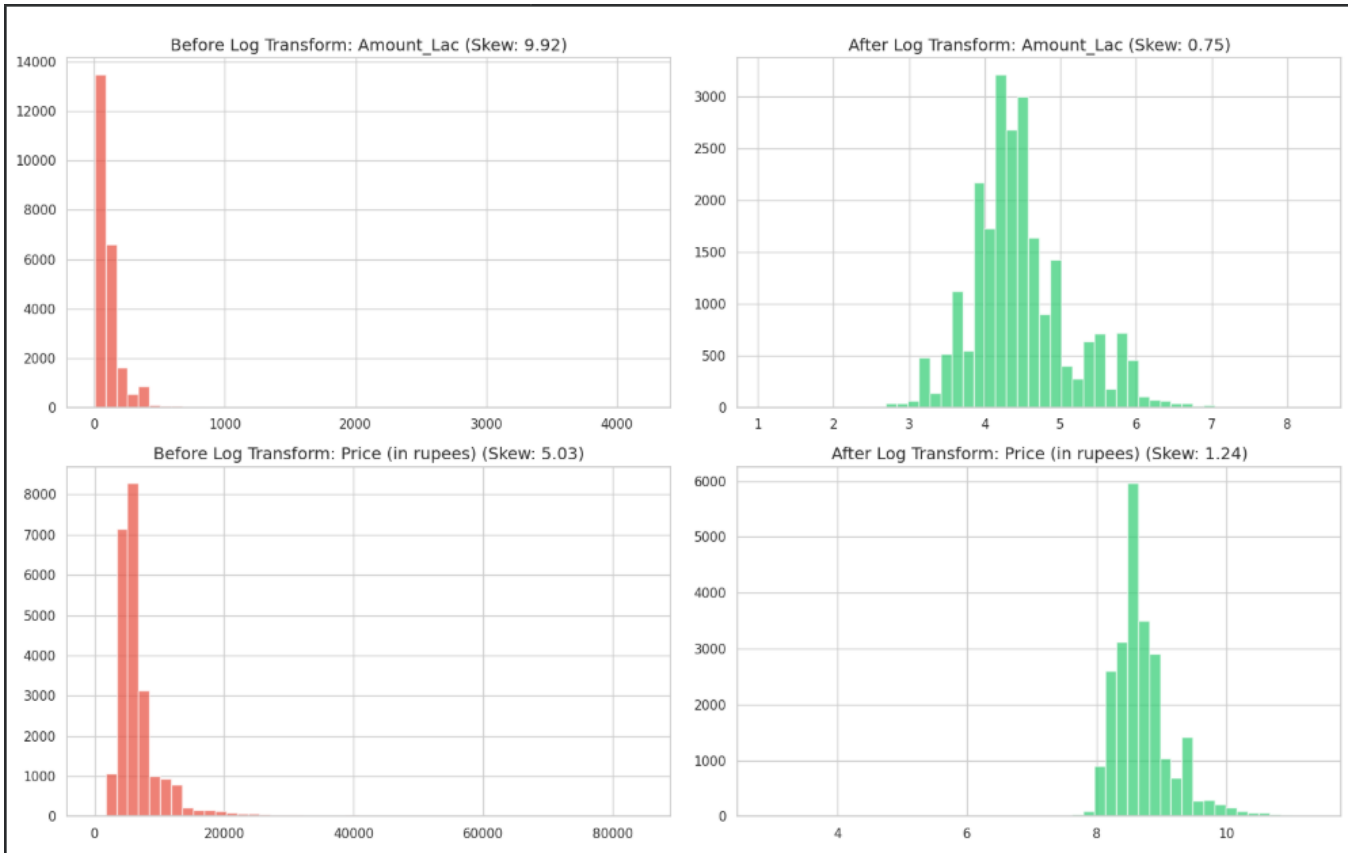
6.3 Handling Skewness

Price and area columns are heavily right-skewed. We applied log1p transform (log(1+x)) which compresses the long tail and makes the distribution closer to normal. This is useful for statistical methods that assume normality.

Code:

```
skew_cols = ['Amount_Lac', 'Price (in rupees)', 'Carpet_Area_sqft']
skew_cols = [c for c in skew_cols if c in df_encoded.columns]

fig, axes = plt.subplots(len(skew_cols), 2, figsize=(16, 5*len(skew_cols)))
for i, col in enumerate(skew_cols):
    axes[i, 0].hist(df_encoded[col], bins=50, color='#e74c3c', alpha=0.7, edgecolor='white')
    axes[i, 0].set_title(f'Before: {col}  (skew={df_encoded[col].skew():.2f})')

    log_col = np.log1p(df_encoded[col].clip(lower=0))
    axes[i, 1].hist(log_col, bins=50, color='#2ecc71', alpha=0.7, edgecolor='white')
    axes[i, 1].set_title(f'After log1p: {col}  (skew={log_col.skew():.2f})')

plt.suptitle('Skewness Fix - Log Transform', fontsize=16, y=1.02)
plt.tight_layout()
plt.show()
```

Visualization:



6.4 Feature Creation

We created three new derived features that capture information that wasn't directly in the raw data:

Code:

```
df_feat = df_encoded.copy()
```

```
# Price normalized by number of bathrooms
df_feat['Price_per_Bathroom'] = df_feat['Amount_Lac'] / df_feat['Bathroom'].replace(0, 1)

# How high up in the building (0 = ground, 1 = top floor)
df_feat['Floor_Ratio'] = df_feat['Current_Floor'] / df_feat['Total_Floors'].replace(0, 1)

# Combined amenity count
df_feat['Amenity_Score'] = (df_feat['Bathroom']
                           + df_feat['Balcony']
                           + df_feat['Car_Parking_Count'])

print('New features created:')
print(df_feat[['Price_per_Bathroom', 'Floor_Ratio',
'Amenity_Score']].describe().to_string())
```

Output:

```
...  New features created:
       Price_per_Bathroom   Floor_Ratio  Amenity_Score
count        23558.000000  23558.000000   23558.000000
mean            42.891693      0.475624       5.218015
std             37.523927      0.283923      13.068454
min              1.950000      0.000000       2.000000
25%             26.000000      0.250000       4.000000
50%             34.000000      0.428571       5.000000
75%             45.000000      0.700000       5.000000
max           1000.000000      1.000000     711.000000
```

- Price_per_Bathroom – gives a size-adjusted value metric
- Floor_Ratio – captures relative floor position, which can affect price in high-rise buildings
- Amenity_Score – a single score combining bathroom, balcony, and parking counts

Team Contribution:

Name	Roll No.	Contribution
Hariharan V S	24BAD030	Code Preparing
Jessica Sam B	24BAD049	Report Making
Jeevaprakash S	24BAD048	Updating GitHub
Kavin T	24BAD058	Code Preparing

WEEK 7 – Dimensionality Reduction

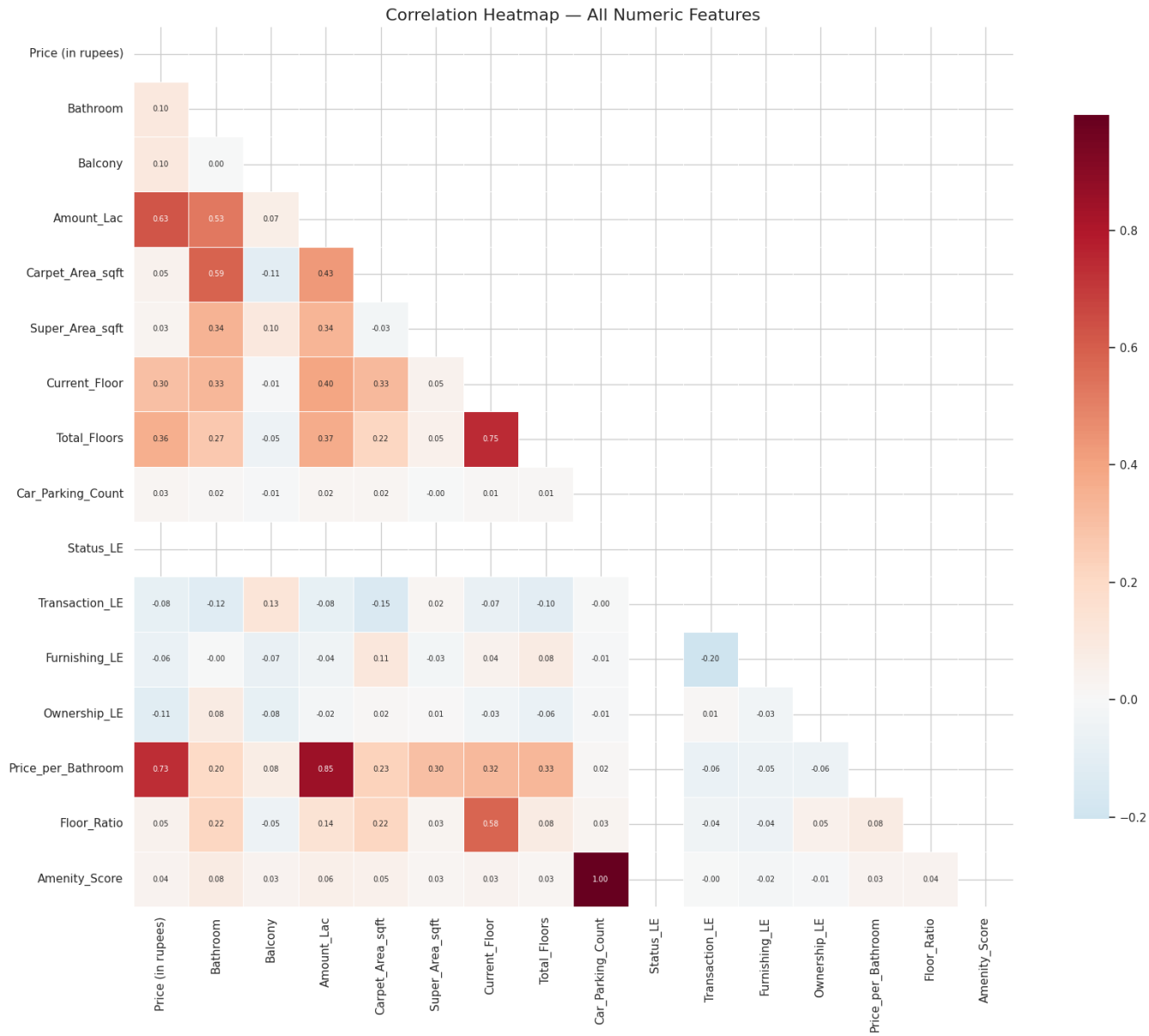
This week we did correlation analysis to understand which features are actually linked to price, then used PCA to reduce the dataset to its most informative dimensions. It's useful to know the effective complexity of the data before building any model.

7.1 Correlation Heatmap

Code:

```
numeric_df = df_feat.select_dtypes(include=[np.number])
corr_matrix = numeric_df.corr()

fig, ax = plt.subplots(figsize=(18, 14))
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
sns.heatmap(corr_matrix, mask=mask, annot=True, fmt='.2f',
            cmap='RdBu_r', center=0, square=True,
            linewidths=0.5, cbar_kws={'shrink': 0.8},
            ax=ax, annot_kws={'size': 7})
ax.set_title('Correlation Heatmap - All Numeric Features', fontsize=16)
plt.tight_layout()
plt.show()
```

Visualization:



Key correlations we noticed:

- Amount_Lac ↔ Carpet_Area_sqft – strong positive. Makes intuitive sense.
- Amount_Lac ↔ Bathroom – moderate positive. More bathrooms usually means a bigger and pricier flat.
- Amenity_Score ↔ Amount_Lac – meaningful positive correlation.
- Price per sqft doesn't correlate strongly with area — it captures location premium more than size.

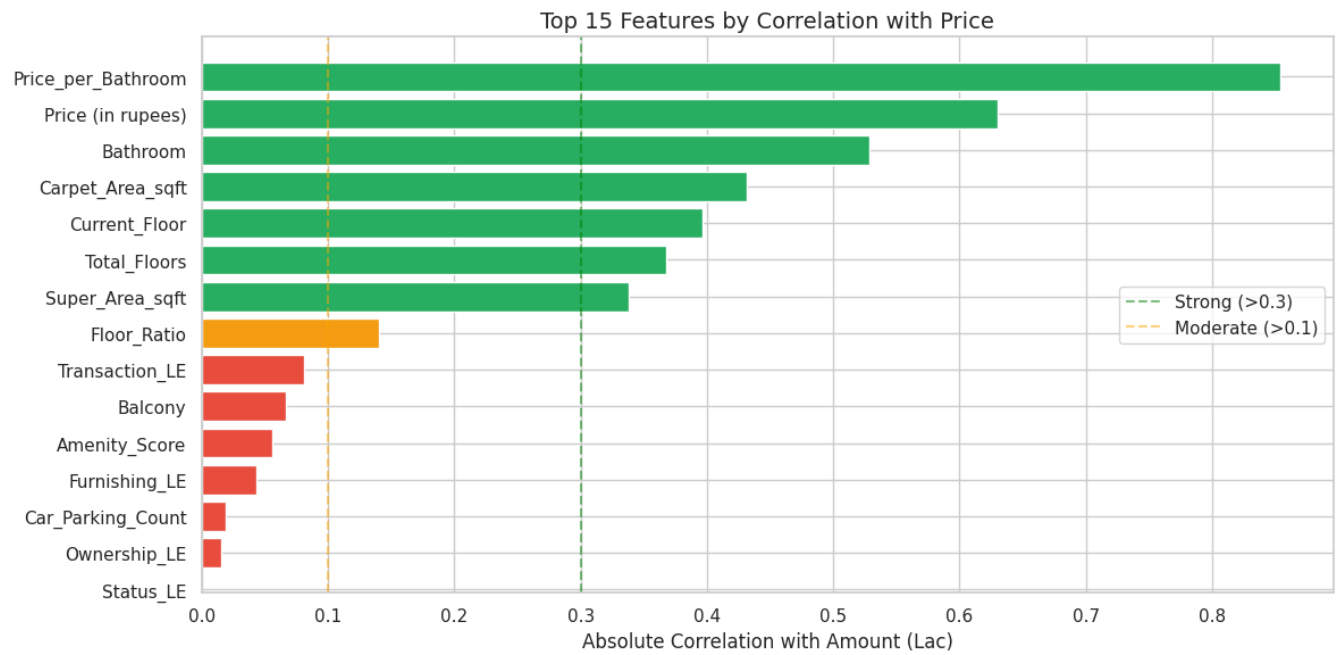
7.2 Feature Selection – Top Predictors

Code:

```
if 'Amount_Lac' in corr_matrix.columns:
    target_corr =
corr_matrix['Amount_Lac'].drop('Amount_Lac').abs().sort_values(ascending=False)

    fig, ax = plt.subplots(figsize=(12, 6))
```

```
    colors = ['#27ae60' if x > 0.3 else '#f39c12' if x > 0.1 else '#e74c3c'
              for x in target_corr.values]
    ax.barh(target_corr.index[:15], target_corr.values[:15], color=colors[:15])
    ax.set_xlabel('Absolute Correlation with Price (Amount_Lac)')
    ax.set_title('Top 15 Features by Correlation with Price')
    ax.invert_yaxis()
    ax.axvline(x=0.3, color='green', linestyle='--', alpha=0.5, label='Strong (>0.3)')
    ax.axvline(x=0.1, color='orange', linestyle='--', alpha=0.5, label='Moderate (>0.1)')
    ax.legend()
    plt.tight_layout()
    plt.show()

    print('Top 10 features:')
    print(target_corr.head(10).to_string())
```

Visualization:




```
Top features for predicting price:
Price_per_Bathroom    0.853449
Price (in rupees)     0.630344
Bathroom              0.528196
Carpet_Area_sqft      0.431529
Current_Floor         0.396847
Total_Floors          0.368149
Super_Area_sqft       0.338224
Floor_Ratio           0.140545
Transaction_LE        0.080640
Balcony               0.066366
```

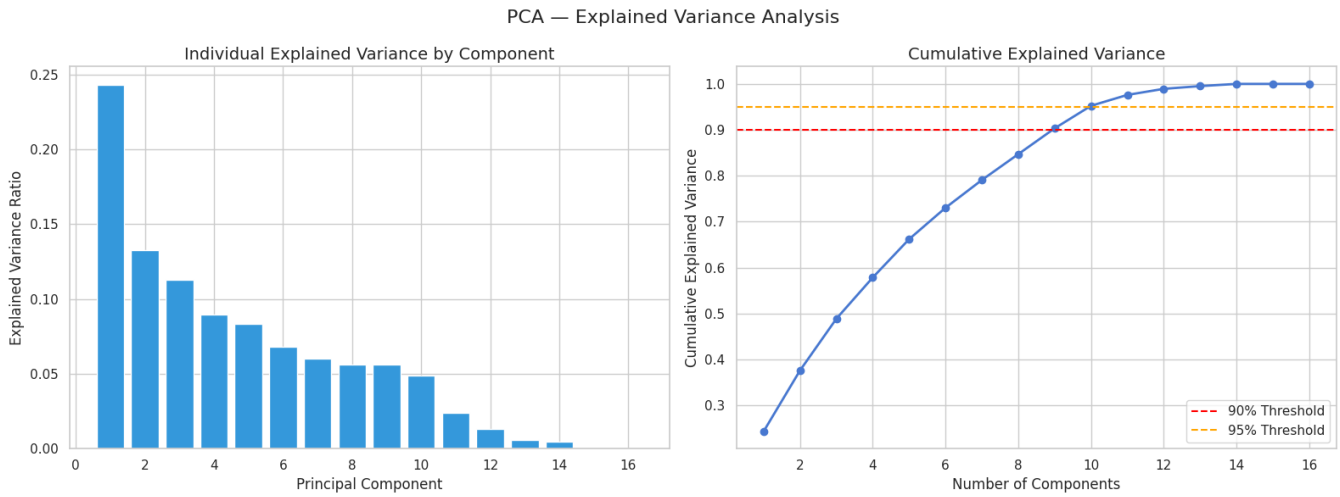
Carpet Area, Bathroom count, and Amenity Score came out on top. These should be the primary features for any future prediction model.

7.3 Principal Component Analysis (PCA)

Code – Explained Variance:

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

pca_features = numeric_df.dropna()
pca_scaler   = StandardScaler()
pca_data     = pca_scaler.fit_transform(pca_features)

pca_full = PCA()
pca_full.fit(pca_data)

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Individual variance per component
axes[0].bar(range(1, len(pca_full.explained_variance_ratio_)+1),
            pca_full.explained_variance_ratio_, color='#3498db', edgecolor='white')
axes[0].set(xlabel='Component', ylabel='Explained Variance Ratio',
            title='Variance per Component')

# Cumulative
cumulative = np.cumsum(pca_full.explained_variance_ratio_)
axes[1].plot(range(1, len(cumulative)+1), cumulative, 'bo-', linewidth=2)
axes[1].axhline(y=0.90, color='red',    linestyle='--', label='90%')
axes[1].axhline(y=0.95, color='orange', linestyle='--', label='95%')
axes[1].set(xlabel='Number of Components', ylabel='Cumulative Variance',
            title='Cumulative Explained Variance')
axes[1].legend()

plt.suptitle('PCA - Explained Variance', fontsize=16)
plt.tight_layout()
plt.show()

n_90 = np.argmax(cumulative >= 0.90) + 1
n_95 = np.argmax(cumulative >= 0.95) + 1
print(f'Components for 90% variance: {n_90}')
print(f'Components for 95% variance: {n_95}')
```

Visualization:

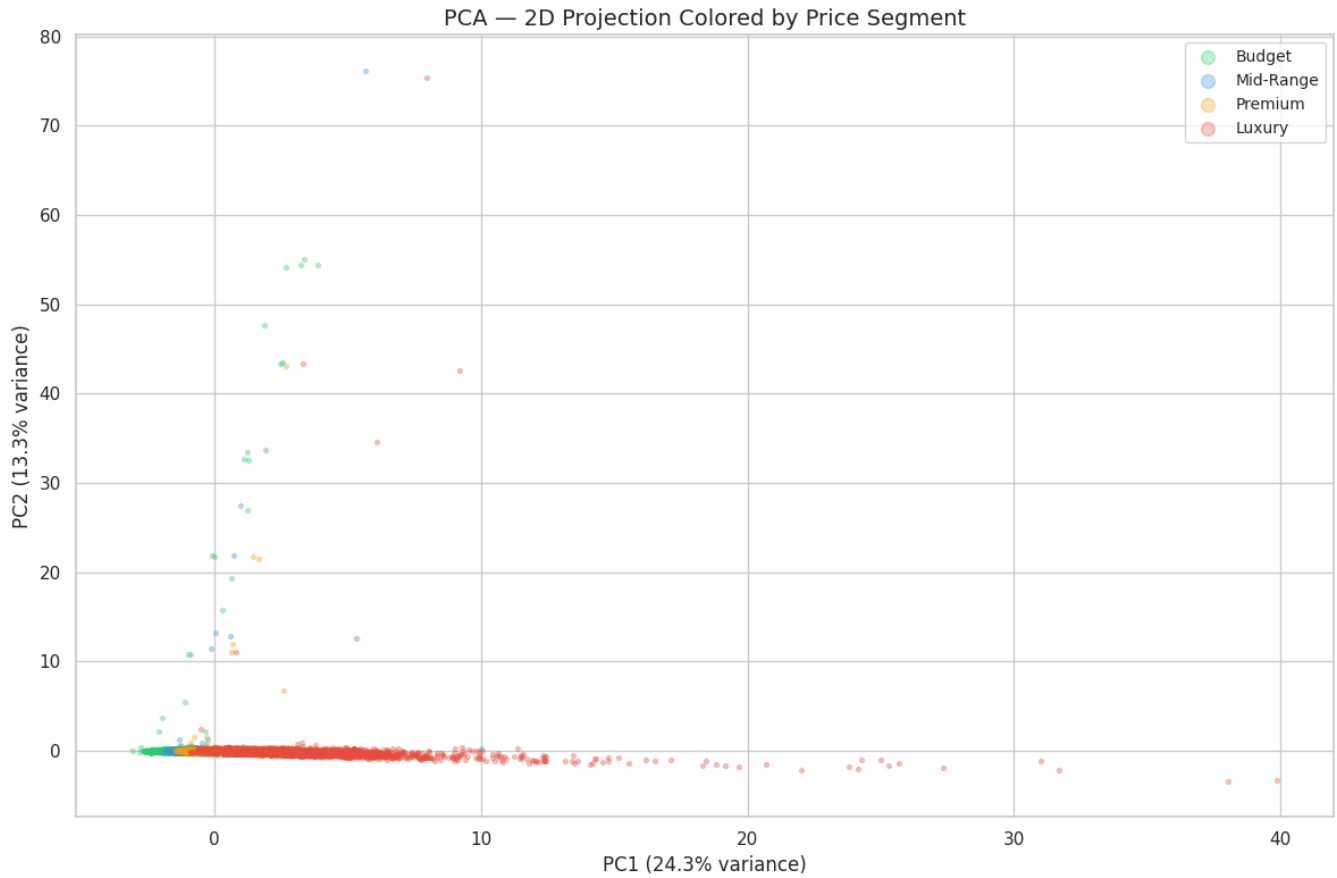



```
Components for 90% variance: 9
Components for 95% variance: 10
```

Code – 2D PCA Plot:

```
pca_2d     = PCA(n_components=2)
pca_result = pca_2d.fit_transform(pca_data)

amount_col = pca_features['Amount_Lac'] if 'Amount_Lac' in pca_features.columns else
pca_features.iloc[:, 0]
quartiles  = pd.qcut(amount_col, q=4, labels=['Budget', 'Mid-Range', 'Premium', 'Luxury'])

fig, ax = plt.subplots(figsize=(12, 8))
colors_map = {'Budget': '#2ecc71', 'Mid-Range': '#3498db', 'Premium': '#f39c12', 'Luxury':
'#e74c3c'}
for label in ['Budget', 'Mid-Range', 'Premium', 'Luxury']:
    mask = quartiles == label
    ax.scatter(pca_result[mask, 0], pca_result[mask, 1],
               alpha=0.3, s=8, c=colors_map[label], label=label)

ax.set(xlabel=f'PC1 ({pca_2d.explained_variance_ratio_[0]*100:.1f}% variance)',
       ylabel=f'PC2 ({pca_2d.explained_variance_ratio_[1]*100:.1f}% variance)',
       title='PCA 2D - Coloured by Price Segment')
ax.legend(markerscale=3, fontsize=10)
plt.tight_layout()
plt.show()
```

Visualization:



What we interpreted from PCA:

- Only a handful of components are needed to capture 90%+ of the variance — the data is less complex than its 21 features suggest.
- PC1 clearly separates Budget and Luxury segments — it's capturing overall property size/value.
- PC2 likely captures location premium or amenity differences.
- Mid-range properties overlap a lot, which makes them harder to distinguish on features alone.

Team Contribution:

Name	Roll No.	Contribution
Hariharan V S	24BAD030	Code Preparing & PCA Implementation
Jessica Sam B	24BAD049	Report Making
Jeevaprakash S	24BAD048	Updating GitHub
Kavin T	24BAD058	Correlation Analysis & Feature Selection

WEEK 8 – Dashboard Construction

Tool Used

Power BI

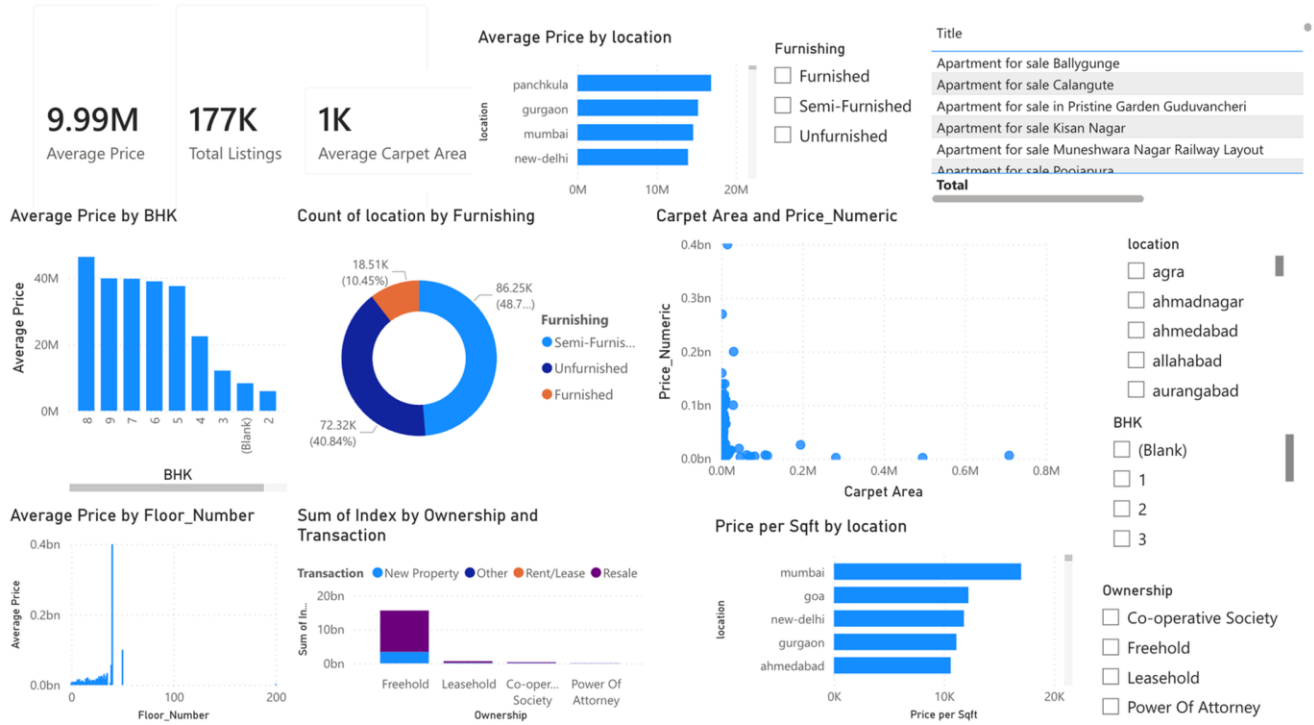
Types of Visualizations Used

- KPI Cards
- Clustered Column Chart
- Bar Chart
- Donut Chart
- Scatter Plot
- Column Chart
- Stacked Column Chart
- Slicers (Filters)

Dashboard Explanation

This dashboard provides a comprehensive analysis of real estate properties based on various attributes such as price, location, area, furnishing status, ownership, and floor level. The objective is to identify pricing trends, property distribution, and key factors influencing real estate value.

Dashboard (Screenshot):



Insights and Storytelling

Key Findings from Analysis

- Average property price is approximately 9.99 million, indicating a high-value market.

- Dataset has ~177,000 usable listings after cleaning.
- Average carpet area is around 1,000 sq.ft — most properties are mid-sized.
- Location is a major factor — Panchkula, Gurgaon, Mumbai, New Delhi show higher average prices.
- Semi-furnished properties make up the largest share (~48%), followed by unfurnished (~40%) and furnished (~10%).
- Positive correlation between carpet area and price, though outliers exist.

Patterns and Trends:

- Higher BHK (6–8) = significantly higher prices. Lower BHK (1–3) = more affordable and dominate volume.
- Metro cities show higher price per sqft — high demand, limited space.
- Freehold properties dominate — buyers prefer full ownership rights.
- Certain floor levels show price spikes — preference for higher floors in apartment complexes.

Business/Social Insights:

- Location is the single biggest factor in pricing.
- Semi-furnished homes are preferred for the move-in flexibility they offer.
- BHK-price relationship shows clear market segmentation: lower BHK = mass market, higher BHK = premium/luxury.

Decision-Making Recommendations:

- For Buyers – compare price per sqft, not just total price. Emerging cities offer better value.
- For Sellers/Builders – invest in semi-furnished properties, target metro cities for premium projects.
- For Real Estate Companies – use location-based pricing strategies, focus marketing on mid-range BHK.
- For Modelling – use top correlated features, apply PCA to reduce dimensions, handle outliers carefully.

Name	Roll No.	Contribution
Hariharan V S	24BAD030	Dashboard
Jessica Sam B	24BAD049	Storytelling
Jeevaprakash S	24BAD048	Report making
Kavin T	24BAD058	Insights